SiRAT Station API: Design Document

Oct 27, 2026 · @Shoyon Kermany

Overview

SiRAT becomes a thin PyQt5 client of one typed API that runs in the station service, inside the workspace. The functions already exist in other libraries; this document defines the API layer that exposes them, with typed inputs, outputs, models and errors.

Rule: one API method per user intent, not per library function. "Connect DUT" is one call even if `BenchUtils` needs several steps. The GUI never imports `BenchUtils`, `SwitchUtils` or `dut_control`.

In scope

- The API contract for the SiRAT features that touch hardware.
- Facts and `requires` rules that decide when a call is allowed.
- Events for polling, progress and disconnects.
- Package layout and the first vertical slice.

Out of scope

- Remote execution through the hub (later).
- Workspace migration, package structure and loopback setup (done).
- Features that never touch hardware: favorites, notes, formulas, history, search and settings stay in the GUI.

Decisions already made

- Transport is TCP on loopback.
- The GUI ships in the same workspace as the service, so both import one `contracts` package.
- The service runs test code and SiRAT calls in its own process; there is no child runner. Test classes use pandas, so pandas and the NI drivers share the service process.
- State is a set of facts per DUT position plus `requires` on each method. Only long operations use a state machine.

Architecture

Four layers. Each only imports the one above it in this list.

1. **Contracts**: typed models and method declarations. No hardware imports. The GUI imports only this.
2. **Domain services**: one per domain. Thin adapters over the existing libraries.
3. **Server**: TCP listener, framing, router, `requires` check, event bus, activity machine.
4. **Client SDK**: typed proxy built from the contracts, plus a worker thread that turns replies and events into Qt signals.

Package layout

```
station_platform/
  contracts/
    common.py      # ids, Position, errors
    dut.py  registers.py  radio.py
    rf.py   instruments.py
    automation.py  advanced.py
    sys.py
  domains/
    dut/  registers/  radio/  ...
  server/
    router.py  requires.py  events.py
    activity.py  transport.py
  client/
    proxy.py  worker.py  qt.py
  gui/            # SiRAT
```

Dependency rules

- `gui` imports `client` and `contracts`. Never `domains`.
- `contracts` imports nothing but pydantic and the standard library.
- `domains` import `contracts` and the hardware libraries.
- A CI check fails the build if `gui` or `contracts` transitively import NI drivers or pandas.

One process on the service side

Domain services, test code and `py.exec` all run in the service process. Calls from one client are handled on a small pool. Calls that touch the same DUT position are serialized by a per-position lock. Instrument calls take the instrument's own lock.

Release coupling

The GUI ships in the same workspace release as the service. `sys.hello` returns `release_id` and `protocol_version`. A mismatch is reported to the user, and the swap happens when the station is idle.

Protocol

Transport

- TCP on `127.0.0.1`. Port from the workspace config, default `47800`.
- Bind to loopback only.
- One connection carries requests, replies and pushed events.
- The client reconnects with backoff. On reconnect it calls `sys.hello` and re-subscribes.

Framing

- 4-byte big-endian length, then a UTF-8 JSON body.
- Max frame 16 MB. Larger payloads (captures, logs) use a paged method.

Request

```
{
  "id": 17,
  "method": "registers.read",
  "params": { "position": 1,
              "name": "CMU.CTRL" },
  "timeout_ms": 5000
}
```

Reply (ok)

```
{ "id": 17, "ok": true,
  "result": { "value": 4660 } }
```

Reply (error)

```
{ "id": 17, "ok": false,
  "error": {
    "code": "wrong_state",
    "message": "DUT 1 is not connected",
    "data": { "missing": ["dut_connected"] }
  } }
```

Event (no id)

```
{ "event": "registers.watch",
  "seq": 912,
  "data": { "sub": "w1", "values": [] } }
```

Rules

- Replies may arrive out of order. The `id` matches them up.
- Event `seq` is per connection and increases by one. A gap means the client missed events and must refetch with `sys.status`.
- `timeout_ms` defaults to the RPC timeout setting (5 s, range 1 to 300 s). The service answers `timeout` and keeps the hardware call to completion in the background.
- Long operations return an `activity_id` at once. Progress and the result arrive as `activity.*` events.
- `sys.describe` returns every method with its JSON schemas and `requires`. The client proxy is built from it, and from the Python contracts when both sides share the package.

State: facts and requires

No session state machine. The service keeps plain facts per DUT position. Each method declares what it `requires`.

Facts per position (1 to 4)

- `dut_connected`
- `phy_loaded` (name of the PHY, or none)
- `radio_configured` (rx, tx)
- `watching` (watch subscription active)
- `activity` (idle, starting, executing, aborting, done)
- `fault` (none, or a cause)

Facts per station

- `instruments.<id>.connected`
- `rf.path` per position
- `claim` (who holds the hardware)

Requires tags

Tag	Meaning
`dut`	`dut_connected`
`phy`	`phy_loaded`
`radio_rx`	rx configured
`radio_tx`	tx configured
`instrument:<id>`	that instrument connected
`idle`	`activity` is idle
`no_fault`	`fault` is none

A method lists tags, for example `radio.start_rx` requires `dut`, `phy`, `radio_rx`, `idle`, `no_fault`.

Enforcement

- The router checks `requires` before calling the service.
- If any tag is unmet the call fails with `wrong_state` and lists the missing tags.
- `sys.status` returns the facts.
- The GUI enables buttons from the same declaration and the last `sys.status`. The service still enforces, so a stale GUI is safe.
- All checks live in one function, `check_requires(method, facts)`.

Reset rules

- DUT disconnect clears `phy_loaded`, `radio_configured`, `watching`. It stops any rx or tx.
- Loading a PHY clears `radio_configured`.
- Reconfiguring rx while running needs a stop first, otherwise `wrong_state`.
- An instrument drop clears `radio_configured` for anything routed through it.

- A hardware error during an activity sets `fault`. `sys.recover` applies the safe state and clears it.

Activity machine

Used only for long operations such as sequences, captures and characterization runs.

```
idle -> starting -> executing -> done
                        |
                        v
                    aborting -> idle
```

- Abort is cooperative at the next wrapped hardware call and ends in the safe state.
- A stuck run is ended after a grace period by restarting the service. The run is marked failed with cause `service_restart`.
- One activity per position at a time.

Conventions and models

Conventions

- Pydantic v2. Every request and result is a model, never a bare dict.
- Method names are `domain.verb`, snake case.
- Every per-DUT method takes `position: Position` (an int, 1 to 4).
- Models are frozen. Unknown fields are rejected.
- Enums are string enums, so the wire stays readable.
- Wire models are not GUI view-models. The GUI maps them.
- Values that can exceed 32 bits are ints on the wire. Raw bytes are base64.

Declaring a method

```
@method("registers.read",
        requires=["dut"])
def read(req: ReadReq) -> ReadRes: ...
```

The decorator registers the schema and `requires`. The server and the client proxy both read it.

Common models (`contracts/common.py`)

```
Position = Annotated[int, Field(ge=1, le=4)]

class Empty(Model): pass

class ActivityRef(Model):
    activity_id: str

class Fault(Model):
    cause: str
    detail: str
    at: datetime

class PositionFacts(Model):
    position: Position
    dut_connected: bool
    phy: str | None
    rx_configured: bool
    tx_configured: bool
    watching: bool
    activity: ActivityState
    fault: Fault | None

class InstrumentFacts(Model):
    id: str
    kind: InstrumentKind
    connected: bool

class StationStatus(Model):
    release_id: str
    positions: list[PositionFacts]
    instruments: list[InstrumentFacts]
    claim: str | None
```

Enums

- `ActivityState`: idle, starting, executing, aborting, done.
- `InstrumentKind`: signal_generator, spectrum_analyzer, rf_switch, chamber, eca, power_supply.
- `Access`: ro, wo, rw.
- `BusKind`: rpc, swd, uart.

Register models (`contracts/registers.py`)

```
class FieldInfo(Model):
    name: str
    lsb: int
    width: int
    access: Access

class RegisterInfo(Model):
    name: str          # CMU.CTRL
    address: int
    width: int
    access: Access
    critical: bool     # needs confirmation
    read_sensitive: bool
    fields: list[FieldInfo]
```

Error models (`contracts/common.py`)

```
class ApiError(Model):
    code: ErrorCode
    message: str
    data: dict[str, Any] = {}
```

The full error list is in the Events and errors section.

API by domain

Each entry: method, input, output, `requires`. `pos` means `position: Position`. Anything not listed in `requires` is not needed.

sys

```
sys.hello(client, protocol)
  -> release_id, protocol,
     server_time
  requires: none

sys.describe()
  -> methods[]: name, req_schema,
     res_schema, requires

sys.status()
  -> StationStatus

sys.subscribe(topics[])
  -> Empty
  topics: status, activity,
          watch, instrument, log

sys.recover(pos)
  -> Empty   (safe state, clear fault)
```

dut

```
dut.connect(pos, config_type,
            config_file)
  -> ConnectRes: chip, rpc_ok
  requires: no_fault

dut.disconnect(pos)
  -> Empty

dut.info(pos)
  -> DutInfo: hw{}, rta{}, chip,
     revision
  requires: dut

dut.load_phy(pos, phy: PhySpec)
  -> LoadPhyRes: name
  requires: dut, idle

dut.reset(pos)
```

```
  -> Empty
  requires: dut, idle

dut.set_rpc_timeout(seconds)
  -> Empty
```

`PhySpec` is one of `ble(name)` or `wifi(config_name)`. The service picks `reset_and_download_lpw_phy` or `load_wifi_phy`.

registers

```
registers.tree(pos)
  -> peripherals[]: name,
     registers[RegisterInfo]
  requires: dut

registers.read(pos, name)
  -> value, fields{}
  requires: dut

registers.read_many(pos, names[])
  -> items[]: name, value | error
  requires: dut

registers.write(pos, name, value,
    confirm=false, verify=true)
  -> value_written, value_readback
  requires: dut, idle

registers.write_field(pos, name,
    field, value, confirm, verify)
  -> same as write
  requires: dut, idle

registers.watch(pos, names[],
                interval_s)
  -> sub_id
  requires: dut

registers.unwatch(sub_id)
  -> Empty
```

- `confirm` must be true for critical peripherals (CMU, EMU, MSC, SMU), else `confirmation_required`.
- Read-sensitive peripherals (USART, I2C, SPI) are refused by `read_many` and `watch` with `read_sensitive`.
- `interval_s` range 0.1 to 60.

radio

```
radio.configure_rx(pos, RxConfig)
  -> Empty
  requires: dut, phy, idle

radio.configure_tx(pos, TxConfig)
  -> Empty
  requires: dut, phy, idle

radio.start_rx(pos)
  -> ActivityRef
  requires: dut, phy, radio_rx,
            idle, no_fault

radio.start_tx(pos, mode)
  -> ActivityRef
  requires: dut, phy, radio_tx,
            idle, no_fault

radio.stop(pos)
  -> Empty

radio.read_rssi(pos)
  -> rssi_dbm
  requires: dut, radio_rx

radio.packet_stats(pos)
  -> sent, received, errors
  requires: dut
```

`mode` is `continuous`, `packet` or `gfsk_packet`.

rf

```
rf.paths()
  -> names[]

rf.connect(path, pos)
  -> Empty
  requires: instrument:rf_switch

rf.disconnect(pos)
  -> Empty

rf.current(pos)
  -> path | null

rf.prs_route(pos, PrsRoute)
  -> Empty
  requires: dut
```

instruments

```
instruments.list()
  -> InstrumentFacts[]

instruments.connect(id, address)
  -> Empty

instruments.disconnect(id)
  -> Empty

instruments.siggen.set(id, freq_hz,
    level_dbm, role)
  -> Empty
  requires: instrument:<id>

instruments.siggen.trigger(id, role)
  -> Empty
  role: InfiniteTrigger,
    SingleTrigger, RfOn, RfOff

instruments.chamber.set_temp(id,
    celsius, wait_stable=true)
```

```
  -> ActivityRef

instruments.eca.capture(id, CaptureSpec)
  -> ActivityRef
```

automation

```
automation.actions.list()
  -> Action[] (from the station)

automation.seq.run(pos, ops[],
    stop_on_error=true)
  -> ActivityRef
  requires: dut, idle, no_fault

automation.seq.abort(activity_id)
  -> Empty

automation.seq.status(activity_id)
  -> state, step, results[]
```

`ops[]` is a list of `Op`. Each op is a tagged union: `register_write`, `field_write`, `custom_function`, `rf_switch`, `signal_generator`, `load_phy`, `load_wifi_phy`, `dut_control_method`, `dut_reconnect`, `sequence`, `prs_route`, `gfsk_packet`. Results are one per op with `ok`, `value`, `error`.

advanced

```
advanced.py_exec(pos, code, mode)
  -> stdout, stderr, result_repr
  requires: dut

advanced.dut_control_describe(pos)
  -> modules[]: path, methods[]:
     name, params[], doc
  requires: dut

advanced.dut_control_invoke(pos,
    path, args, kwargs)
  -> result_repr
  requires: dut, idle

advanced.custom_function(pos, name,
    args)
  -> result_repr
  requires: dut, idle
```

- `mode` is `eval` or `exec`. Code runs on the service with `dut` bound to that position.
- `result_repr` is a string. Anything that is not JSON-safe is returned as its `repr`.
- Calls from here are logged with the client id.

Events and errors

Events

Pushed on the same connection after `sys.subscribe`. Each has a per-connection `seq`.

```
status.changed
  data: StationStatus
  when: any fact changes

activity.progress
  data: activity_id, pos,
        step, total, message

activity.finished
  data: activity_id, pos, ok,
```

```
        result | error

registers.watch
  data: sub_id, pos, ts,
        values[{name, value}]

dut.disconnected
  data: pos, cause, detected_by

instrument.changed
  data: InstrumentFacts

log.line
  data: ts, level, source, text
```

Notes

- `registers.watch` is batched. One event per interval per subscription, so a 0.1 s watch is 10 events a second at most.
- If the client is slow, the server drops older `registers.watch` events for that subscription and keeps the newest. Other events are never dropped.
- `dut.disconnected` replaces the client-side error-string matching. Disconnect is declared after the 2 s debounce.
- `activity.finished` is sent exactly once per activity, including after an abort or `service_restart`.

Errors

Every failure is an `ApiError` with a `code`. Codes are an enum in `contracts/common.py`.

Code	Meaning
`wrong_state`	`requires` unmet. `data.missing` lists tags
`invalid_params`	schema validation failed
`unknown_method`	not in `sys.describe`
`timeout`	call exceeded `timeout_ms`
`busy`	position or instrument locked
`claimed`	another owner holds the station

Code	Meaning
`confirmation_required`	critical register, no `confirm`
`read_sensitive`	register is read sensitive
`access_denied`	register is not readable or writable
`out_of_range`	value fails the range check
`verify_failed`	read-back differs from written value
`not_found`	register, path or id unknown
`dut_disconnected`	link lost during the call
`instrument_error`	instrument reported an error
`hardware_error`	library raised. Sets `fault`
`aborted`	activity aborted
`internal`	bug. Includes a trace id

Per-domain errors

Each domain may add detail in `data`, for example `verify_failed` carries `written` and `readback`. The code set stays small and shared so the GUI has one error handler.

Mapping rules

- Library exceptions are caught at the domain service boundary and mapped to a code.
- Raw library messages go in `message`. The GUI shows its own wording per code.
- `hardware_error` and `dut_disconnected` update facts before the reply is sent, so the next `sys.status` is already right.

SiRAT feature mapping

Where each SiRAT feature lands. "GUI" means it stays client-side and never calls the API.

Connection

- Connect and disconnect DUT: `dut.connect`, `dut.disconnect`
- Disconnect detection: `dut.disconnected` event
- RPC timeout setting: `dut.set_rpc_timeout`

- System info panel: `dut.info`
- Load BLE or WiFi PHY: `dut.load_phy`

Registers

- Peripheral tree: `registers.tree` once, then the GUI caches it. SVD browsing and search are GUI.
- Read and write: `registers.read`, `registers.write`, `registers.write_field`
- Access policy and confirmation: enforced in the service. The GUI shows the dialog on `confirmation_required`.
- Write read-back verification: `verify` flag, `verify_failed`
- Watch list: `registers.watch` plus `registers.watch` events
- Data logger: same subscription. The GUI writes the file.
- Favorites, notes, formulas, history: GUI

Radio and RF

- Rx and Tx setup and run: `radio.*`
- RF switch path to DUT: `rf.connect`
- Signal generator roles: `instruments.siggen.*`
- Chamber: `instruments.chamber.set_temp`
- ECA capture: `instruments.eca.capture`
- PRS routing: `rf.prs_route`

Automation

- Actions list: `automation.actions.list`
- Sequence editor: GUI builds `ops[]`
- Run, abort, progress: `automation.seq.*` and `activity.*` events

Advanced

- Python console: `advanced.py_exec`
- Custom functions: `advanced.custom_function`
- DUT control API explorer: `advanced.dut_control_describe` and `advanced.dut_control_invoke`

GUI only

- `~/.sirat` settings, window layout, themes

- Favorites, notes, formulas, command history
- SVD parsing and search

Debug mode

SiRAT used to start from a PyCharm debug console paused at a test breakpoint, using that test's `dut`. With the service, the debugger must suspend only the thread ("Suspend: Thread"), otherwise the service cannot answer. The client attaches to the position the test has connected and does not call `dut.connect`.

Gaps to confirm

- Whether the chamber and ECA are used by SiRAT directly or only from sequences.
- Whether `dut_control` methods need argument types beyond JSON. Today they are passed as JSON and converted by name.

Testing and first slice

Testing

- **Contract tests**: every model round-trips through JSON. `sys.describe` output is snapshotted, so a contract change shows up as a diff in review.
- **Requires table test**: a table of method, facts and expected result. It runs against `check_requires` only, with no hardware.
- **Invariants**: after any random call sequence, "radio configured implies DUT connected" and "no activity while faulted" hold.
- **Fake domains**: each domain has an in-memory fake with the same contract. The GUI and the router are tested against fakes.
- **Replay**: record real sessions as request and reply logs, then replay them against the router to check `requires` against real behavior.
- **Import check in CI**: `gui` and `contracts` must not import NI drivers or pandas.
- **Service startup check**: pandas and the NI libraries load together in the intended order, or the service refuses to start.
- **Bench smoke test**: connect, read one register, write and verify, start and stop rx, disconnect. Runs on one real station before each release.

First vertical slice

Smallest path that proves the whole stack on real hardware.

1. `contracts/common.py` and `contracts/sys.py`
2. Transport, framing, router and `check_requires`
3. `sys.hello`, `sys.describe`, `sys.status`
4. `dut.connect`, `dut.disconnect`, `dut.info`
5. `registers.tree`, `registers.read`, `registers.write`
6. Client proxy and Qt worker
7. SiRAT connect screen and register view on the API

Done when SiRAT reads and writes a register on one DUT through the service, the connect button enables from `sys.status`, and unplugging the DUT shows a `dut.disconnected` event.

Next slices

1. Watch list, data logger and `registers.read_many`
2. Radio and RF
3. Instruments
4. Sequences and actions
5. Advanced: `py_exec`, DUT control explorer
6. Debug mode

Open questions

1. **Facts plus `requires` is a leaning, not verified.** Write the requires and reset table from the spec, check it against real hardware, and replay recorded sessions. `check_requires` is one function so the model can be swapped.
2. **Debug mode attach.** The contract has no `dut.attach` yet. Does the test hand its connected `dut` to the service, or does SiRAT reconnect?
3. **pandas and NI together.** Does the current harness load both in one process without the msvcp140 conflict? The startup check decides.
4. **One service or several.** One active service per station, or several idle ones? Affects `sys.hello` and the claim model.
5. **Instruments in SiRAT.** Are the chamber and ECA driven from SiRAT screens, or only from sequences?
6. **`dut_control` argument types.** JSON only, or do some methods need typed objects?

7. **Position scope of watches.** One watch subscription per position, or one across positions? The draft is per position.
8. **Auth.** Loopback only with no auth. Is that enough on a shared station, given `advanced.py_exec`?
9. **Docs 1 and 3 and the tickets** still describe the old session state machine. Update them to facts and `requires`?